

Operating Systems and System Programming

Skill-8

Aim

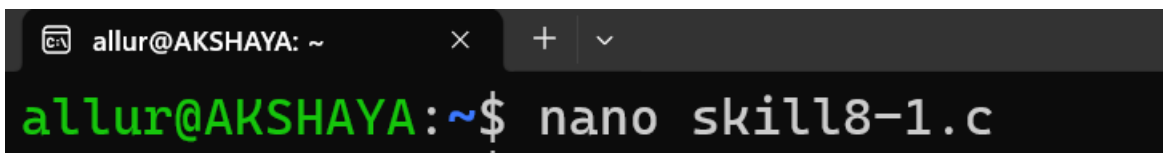
To implement variable expansion in a shell by detecting variable references, expanding values, handling undefined and nested variables, and updating tokens. Also, to identify and execute built-in commands using a dispatch table, handle invalid commands, and maintain shell state.

Algorithm

1. Start the program and read the user's command.
2. Detect variable references in the input and retrieve their values.
3. Expand defined variables and handle undefined variables appropriately.
4. Support nested variable expansion and update the command tokens.
5. Identify built-in commands using a dispatch table.
6. Execute built-in commands within the shell process.
7. Handle invalid commands and display appropriate error messages.
8. Maintain shell state and test variable expansion and command dispatch.
9. Repeat until the user exits the shell.
10. Stop.

Task-1:

Step-1:

A terminal window with a dark background. The title bar shows a window icon, the text 'allur@AKSHAYA: ~', and window control buttons (close, maximize, and a dropdown arrow). The terminal content shows the prompt 'allur@AKSHAYA:~\$' in green, followed by the command 'nano skill8-1.c' in white. A cursor is positioned at the end of the command.

```
allur@AKSHAYA:~$ nano skill8-1.c
```

Step-2:

```
allur@AKSHAYA: ~  
GNU nano 8.7.1 skill8-1.c *  
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
  
void expand(char *input, char *output)  
{  
    int i = 0, j = 0;  
  
    while (input[i] != '\0' && j < 490) {  
        if (input[i] == '$') {  
            char name[100];  
            int k = 0;  
            i++;  
  
            if (input[i] == '{') {  
                i++;  
                while (input[i] != '\0' &&  
                    input[i] != '}' && k < 99)  
                    name[k++] = input[i++];  
                if (input[i] == '}')  
                    i++;  
            }  
        }  
        output[j++] = input[i++];  
    }  
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T E
^X Exit ^R Read File ^\ Replace ^U Paste ^J J

```
allur@AKSHAYA: ~  
GNU nano 8.7.1 skill8-1.c *  
    } else {  
        while ((input[i] >= 'A' && input[i] <= 'Z') ||  
            (input[i] >= 'a' && input[i] <= 'z') ||  
            (input[i] >= '0' && input[i] <= '9') ||  
            input[i] == '_') {  
            if (k < 99)  
                name[k++] = input[i];  
            i++;  
        }  
    }  
  
    name[k] = '\0';  
    const char *value = getenv(name);  
  
    if (value != NULL) {  
        while (*value && j < 490)  
            output[j++] = *value++;  
    }  
} else {  
    output[j++] = input[i++];  
}  
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T E
^X Exit ^R Read File ^\ Replace ^U Paste ^J J

```
allur@AKSHAYA: ~  
GNU nano 8.7.1 skill8-1.c *  
    }  
    }  
    output[j] = '\\0';  
}  
  
int main()  
{  
    char input[500], output[500];  
  
    setenv("NAME", "Linux", 1);  
    setenv("VERSION", "Ubuntu", 1);  
  
    printf("Enter text: ");  
    fgets(input, sizeof(input), stdin);  
  
    expand(input, output);  
  
    printf("Expanded text: %s", output);  
    return 0;  
}|  
  
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T  
^X Exit      ^R Read File  ^\\ Replace    ^U Paste      ^_
```

Step-3:

```
allur@AKSHAYA:~$ gcc skill8-1.c -o skill8-1  
allur@AKSHAYA:~$ ./skill8-1  
Enter text: Hello $NAME  
Expanded text: Hello Linux  
allur@AKSHAYA:~$ ./skill8-1  
Enter text: Welcome to ${VERSION}  
Expanded text: Welcome to Ubuntu  
allur@AKSHAYA:~$ |
```

Task-2:

Step-1:

```
allur@AKSHAYA:~$ nano skill8-2.c
```

Step-2:

```
allur@AKSHAYA: ~  
GNU nano 8.7.1 skill8-2.c *  
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
#include <unistd.h>  
  
void cmd_pwd(char **args)  
{  
    char cwd[1024];  
  
    if (getcwd(cwd, sizeof(cwd)))  
        printf("%s\n", cwd);  
}  
  
void cmd_cd(char **args)  
{  
    if (args[1] == NULL)  
        printf("Usage: cd <directory>\n");  
    else if (chdir(args[1]) != 0)  
        perror("cd");  
}  
  
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T  
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J
```

```
allur@AKSHAYA: ~  
GNU nano 8.7.1 skill8-2.c *  
void cmd_exit(char **args)  
{  
    exit(0);  
}  
  
void cmd_help(char **args)  
{  
    printf("Built-in commands: cd, pwd, help, exit\n");  
}  
  
struct Command {  
    char *name;  
    void (*function)(char **);  
};  
  
int main()  
{  
    struct Command commands[] = {  
        {"cd", cmd_cd},  
        {"pwd", cmd_pwd},  
        {"help", cmd_help},  
    };  
  
    ^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T  
    ^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J
```

```
allur@AKSHAYA: ~ × + v
GNU nano 8.7.1 skill8-2.c *
    {"exit", cmd_exit}
};

int count = sizeof(commands) / sizeof(commands[0]);
char input[100];
char *args[10];

while (1) {
    printf("myshell> ");
    fflush(stdout);

    if (fgets(input, sizeof(input), stdin) == NULL)
        break;

    int n = 0;
    char *token = strtok(input, " \t\n");

    while (token != NULL && n < 9) {
        args[n++] = token;
        token = strtok(NULL, " \t\n");
    }

    ^G Help      ^O Write Out  ^F Where Is   ^K Cut
    ^X Exit      ^R Read File  ^\ Replace    ^U Paste
```

```
allur@AKSHAYA: ~ × + v
GNU nano 8.7.1 skill8-2.c *
    args[n] = NULL;

    if (n == 0)
        continue;

    int found = 0;

    for (int i = 0; i < count; i++) {
        if (strcmp(args[0], commands[i].name) == 0) {
            commands[i].function(args);
            found = 1;
            break;
        }
    }

    if (!found)
        printf("Invalid command: %s\n", args[0]);
}

return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut      ^T E
^X Exit      ^R Read File  ^\ Replace    ^U Paste    ^J J
```

Step-3:

```
allur@AKSHAYA:~$ gcc skill8-2.c -o skill8-2
allur@AKSHAYA:~$ ./skill8-2
myshell> pwd
/home/allur
myshell> help
Built-in commands: cd, pwd, help, exit
myshell> cd ..
myshell> pwd
/home
myshell> hello
Invalid command: hello
myshell> exit
allur@AKSHAYA:~$
```

Result

Variable expansion was implemented to detect references, expand values, handle undefined and nested variables, and update tokens. Built-in command execution was implemented using a dispatch table, with support for invalid commands and maintaining shell state.